

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION COURSE CODE:ITA0515 Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3 Total Marks 30 Duration :60 Minutes Annexure A

Experiments

Code of Conduct:

I S.Jeswanth kumar-192321056 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: S.Jeswanth kumar

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	
---------------	----	---	--	--	----------------------------------	--

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Answers:

1. Harris Corner Detection Analysis

Aim

To detect corner points in an image using the Harris Corner Detection algorithm. Algorithm

1. Import OpenCV.
2. Read the input image.
3. Convert the image to grayscale.
4. Apply the Harris Corner Detection algorithm.
5. Mark the detected corners.

6. Display the output image.

Code:

```
import cv2

img = cv2.imread("Bheem.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = gray.astype("float32")

corners = cv2.cornerHarris(gray, 2, 3, 0.04)

corners = cv2.dilate(corners, None)

img[corners > 0.01 * corners.max()] = [0, 0, 255]

cv2.imshow("Harris Corner Detection", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input:



Output:



Result:

The Harris Corner Detection algorithm successfully detected the corner points in the image and displayed them in red.

2. Effect of Noise on Corner Detection

Aim

To study the effect of noise on Harris Corner

Detection. Algorithm

1. Import OpenCV and NumPy.
2. Read the input image.
3. Convert the image to grayscale.
4. Add noise to the image.
5. Apply Harris Corner Detection.

6. Mark the detected corners.

7. Display the original, noisy, and output images.

Code

```
import cv2

import numpy as np

img = cv2.imread("Bheem.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) noise
= np.random.normal(0, 20, gray.shape).astype(np.uint8)

noisy = cv2.add(gray, noise)

corners = cv2.cornerHarris(noisy.astype("float32"), 2, 3,
0.04) corners = cv2.dilate(corners, None)

result = img.copy()

result[corners > 0.01 * corners.max()] = [0, 255, 0]

cv2.imshow("Original", img)

cv2.imshow("Noisy", noisy)

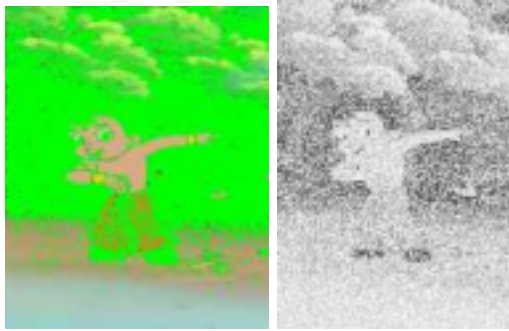
cv2.imshow("Corners", result)
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Input:



Output:



Result:

The noisy image produced fewer and less accurate corner points than the original image, showing that noise affects corner detection performance.

3. Hough Transform for Line Detection

Aim:

To detect straight lines in an image using the Hough Transform.

Algorithm:

1. Import OpenCV and NumPy.

2. Read the input image.
3. Convert the image to grayscale.
4. Detect edges using the Canny edge detector.
5. Apply the Hough Line Transform.
6. Draw the detected lines.
7. Display the output image.

Code:

```
import cv2

import numpy as np

img = cv2.imread("Bheem.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 50, 150)

lines = cv2.HoughLinesP(edges, 1, np.pi/180, 100, minLineLength=50, maxLineGap=10)

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line.reshape(4)
        cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

cv2.imshow("Edges", edges)
cv2.imshow("Hough Lines", img)
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Input:



Output:



Result:

The Hough Transform successfully detected the straight lines in the image and displayed them with green lines.